



Use the R2 multipart API from Workers

By following this guide, you will create a Worker through which your applications can perform multipart uploads. This example worker could serve as a basis for your own use case where you can add authentication to the worker, or even add extra validation logic when uploading each part. This guide also contains an example Python application that uploads files to this worker.

This guide assumes you have set up the [R2 binding](#) for your Worker. Refer to [Use R2 from Workers](#) for instructions on setting up an R2 binding.

An example Worker using the multipart API

The following example Worker exposes an HTTP API which enables applications to use the multipart API through the Worker.

In this example, each request is routed based on the HTTP method and the action request parameter. As your Worker becomes more complicated, consider utilizing a serverless web framework such as [Hono](#)  to handle the routing for you.

The following example Worker includes any new information about the state of the multipart upload in the response to each request. For the request which creates the multipart upload, the `uploadId` is returned. For requests uploading a part, the part number and `etag` are returned. In turn, the client keeps track of this state, and includes the `uploadId` in subsequent requests, and the `etag` and part number of each part when completing a multipart upload.

Add the following code to your project's `index.js` file and replace `MY_BUCKET` with your bucket's name:

JavaScript

```
interface Env {  
  MY_BUCKET: R2Bucket;  
}  
  
export default {  
  async fetch(  
    request,  
    env,  
    ctx  
  ): Promise<Response> {  
    const bucket = env.MY_BUCKET;  
  
    const url = new URL(request.url);  
    const key = url.pathname.slice(1);  
    const action =  
url.searchParams.get("action");  
  
    if (action === null) {  
      return new Response("Missing action  
type", { status: 400 });  
    }  
  
    // Route the request based on the HTTP  
method and action type
```

```
switch (request.method) {
  case "POST":
    switch (action) {
      case "mpu-create": {
        const multipartUpload = await
bucket.createMultipartUpload(key);
        return new Response(
          JSON.stringify({
            key: multipartUpload.key,
            uploadId:
multipartUpload.uploadId,
          })
        );
      }
      case "mpu-complete": {
        const uploadId =
url.searchParams.get("uploadId");
        if (uploadId === null) {
          return new Response("Missing
uploadId", { status: 400 });
        }

        const multipartUpload =
env.MY_BUCKET.resumeMultipartUpload(
          key,
```

```
        uploadId
    );

    interface completeBody {
        parts: R2UploadedPart[];
    }

    const completeBody: completeBody =
await request.json();

    if (completeBody === null) {
        return new Response("Missing or
incomplete body", {
            status: 400,
        });
    }

    // Error handling in case the
multipart upload does not exist anymore

    try {
        const object = await
multipartUpload.complete(completeBody.parts);
        return new Response(null, {
            headers: {
                etag: object.httpEtag,
            },
        });
    }
```

```
    } catch (error: any) {  
        return new  
Response(error.message, { status: 400 });  
    }  
}  
default:  
    return new Response(`Unknown  
action ${action} for POST`, {  
        status: 400,  
    });  
}  
case "PUT":  
    switch (action) {  
        case "mpu-uploadpart": {  
            const uploadId =  
url.searchParams.get("uploadId");  
            const partNumberString =  
url.searchParams.get("partNumber");  
            if (partNumberString === null ||  
uploadId === null) {  
                return new Response("Missing  
partNumber or uploadId", {  
                    status: 400,  
                });  
            }  
        }  
    }  
}
```



```
    if (request.body === null) {
        return new Response("Missing
request body", { status: 400 });
    }

    const partNumber =
parseInt(partNumberString);

    const multipartUpload =
env.MY_BUCKET.resumeMultipartUpload(
        key,
        uploadId
    );
    try {
        const uploadedPart:
R2UploadedPart =
            await
multipartUpload.uploadPart(partNumber,
request.body);
        return new
Response(JSON.stringify(uploadedPart));
    } catch (error: any) {
        return new
Response(error.message, { status: 400 });
    }
}
```

```
        default:
            return new Response(`Unknown
action ${action} for PUT`, {
                status: 400,
            });
        }
        case "GET":
            if (action !== "get") {
                return new Response(`Unknown action
${action} for GET`, {
                    status: 400,
                });
            }
            const object = await
env.MY_BUCKET.get(key);
            if (object === null) {
                return new Response("Object Not
Found", { status: 404 });
            }
            const headers = new Headers();
            object.writeHttpMetadata(headers);
            headers.set("etag", object.httpEtag);
            return new Response(object.body, {
                headers
            });
        case "DELETE":
```

```
switch (action) {
  case "mpu-abort": {
    const uploadId =
url.searchParams.get("uploadId");
    if (uploadId === null) {
      return new Response("Missing
uploadId", { status: 400 });
    }
    const multipartUpload =
env.MY_BUCKET.resumeMultipartUpload(
      key,
      uploadId
    );

    try {
      multipartUpload.abort();
    } catch (error: any) {
      return new
Response(error.message, { status: 400 });
    }
    return new Response(null, {
status: 204 });
  }
  case "delete": {
    await env.MY_BUCKET.delete(key);
```

```
        return new Response(null, {
status: 204 });
    }
    default:
        return new Response(`Unknown
action ${action} for DELETE`, {
            status: 400,
        });
    }
    default:
        return new Response("Method Not
Allowed", {
            status: 405,
            headers: { Allow: "PUT, POST, GET,
DELETE" },
        });
    }
},
} satisfies ExportedHandler<Env>;
```

After you have updated your Worker with the above code, run `npx wrangler deploy`.

You can now use this Worker to perform multipart uploads. You can either send requests from your existing application to this Worker to perform uploads or use a script to upload files through this Worker.

The next section is optional and shows an example of a Python script which uploads a chosen file on your machine to your Worker.

Perform a multipart upload with your Worker (optional)

This example application uploads a local file to the Worker in multiple parts. It uses Python's built-in `ThreadPoolExecutor` to parallelize the uploading of parts to the Worker, which increases upload speeds. HTTP requests to the Worker are made with the [requests](#)  library.

Utilizing the multipart API in this way also allows you to use your Worker to upload files larger than the [Workers request body size limit](#). The uploading of individual parts is still subject to this limit.

Save the following code in a file named `mpuscript.py` on your local machine. Change the `worker_endpoint` variable to where your worker is deployed. Pass the file you want to upload as an argument when running this script: `python3 mpuscript.py myfile`. This will upload the file `myfile` from your machine to your bucket through the Worker.

Python

```
import math
import os
import requests
from requests.adapters import HTTPAdapter,
Retry
import sys
import concurrent.futures

# Take the file to upload as an argument
filename = sys.argv[1]
# The endpoint for our worker, change this to
# wherever you deploy your worker
worker_endpoint =
"https://myworker.myzone.workers.dev/"
# Configure the part size to be 10MB. 5MB is
# the minimum part size, except for the last
# part
partsize = 10 * 1024 * 1024

def upload_file(worker_endpoint, filename,
partsize):
    url = f"{worker_endpoint}{filename}"
```



```
# Create the multipart upload

uploadId = requests.post(url, params=
{"action": "mpu-create"}).json()["uploadId"]

part_count =
math.ceil(os.stat(filename).st_size /
partsize)

# Create an executor for up to 25
concurrent uploads.

executor =
concurrent.futures.ThreadPoolExecutor(25)

# Submit a task to the executor to upload
each part

futures = [
    executor.submit(upload_part, filename,
partsize, url, uploadId, index)
    for index in range(part_count)
]

concurrent.futures.wait(futures)

# get the parts from the futures
uploaded_parts = [future.result() for
future in futures]

# complete the multipart upload
response = requests.post(
```

```
        url,  
        params={"action": "mpu-complete",  
"uploadId": uploadId},  
        json={"parts": uploaded_parts},  
    )  
    if response.status_code == 200:  
        print("🎉 successfully completed  
multipart upload")  
    else:  
        print(response.text)
```

```
def upload_part(filename, partsize, url,  
uploadId, index):  
    # Open the file in rb mode, which treats  
it as raw bytes rather than attempting to  
parse utf-8  
    with open(filename, "rb") as file:  
        file.seek(partsize * index)  
        part = file.read(partsize)  
  
    # Retry policy for when uploading a part  
fails  
    s = requests.Session()
```

```
retries = Retry(total=3, status_forcelist=
[400, 500, 502, 503, 504])

s.mount("https://",
HTTPAdapter(max_retries=retries))

return s.put(
    url,
    params={
        "action": "mpu-uploadpart",
        "uploadId": uploadId,
        "partNumber": str(index + 1),
    },
    data=part,
).json()

upload_file(worker_endpoint, filename,
partsize)
```

State management

The stateful nature of multipart uploads does not easily map to the usage model of Workers, which are

inherently stateless. In a normal multipart upload, the multipart upload is usually performed in one continuous execution of the client application. This is different from multipart uploads in a Worker, which will often be completed over multiple invocations of that Worker. This makes state management more challenging.

To overcome this, the state associated with a multipart upload, namely the `uploadId` and which parts have been uploaded, needs to be kept track of somewhere outside of the Worker.

In the example Worker and Python application described in this guide, the state of the multipart upload is tracked in the client application which sends requests to the Worker, with the necessary state contained in each request. Keeping track of the multipart state in the client application enables maximal flexibility and allows for parallel and unordered uploads of each part.

When keeping track of this state in the client is impossible, alternative designs can be considered. For example, you could track the `uploadId` and which parts have been uploaded in a Durable Object or other database.

Was this helpful?



Previous

← [Use R2 from Workers](#)

Next

[Workers API reference](#) →

Last updated: Aug 13, 2024

Resources

[API](#)

[New to Cloudflare?](#)

[Directory](#)

[Sponsorships](#)

Support

[Help Center](#)

[System Status](#)

[Compliance](#)

[GDPR](#)

Open Source

Company

[cloudflare.com](#)

[Our team](#)

[Careers](#)

Tools

[Cloudflare Radar](#)

[Speed Test](#)

[Is BGP Safe Yet?](#)

[RPKI Toolkit](#)

[Certificate Transparency](#)

Community

 [X](#)

 [Discord](#)

 [YouTube](#)

 [GitHub](#)

© 2025 Cloudflare, Inc. • [Privacy Policy](#) • [Terms of Use](#)

• [Report Security Issues](#) • [Trademark](#) •  [Cookie Preferences](#)